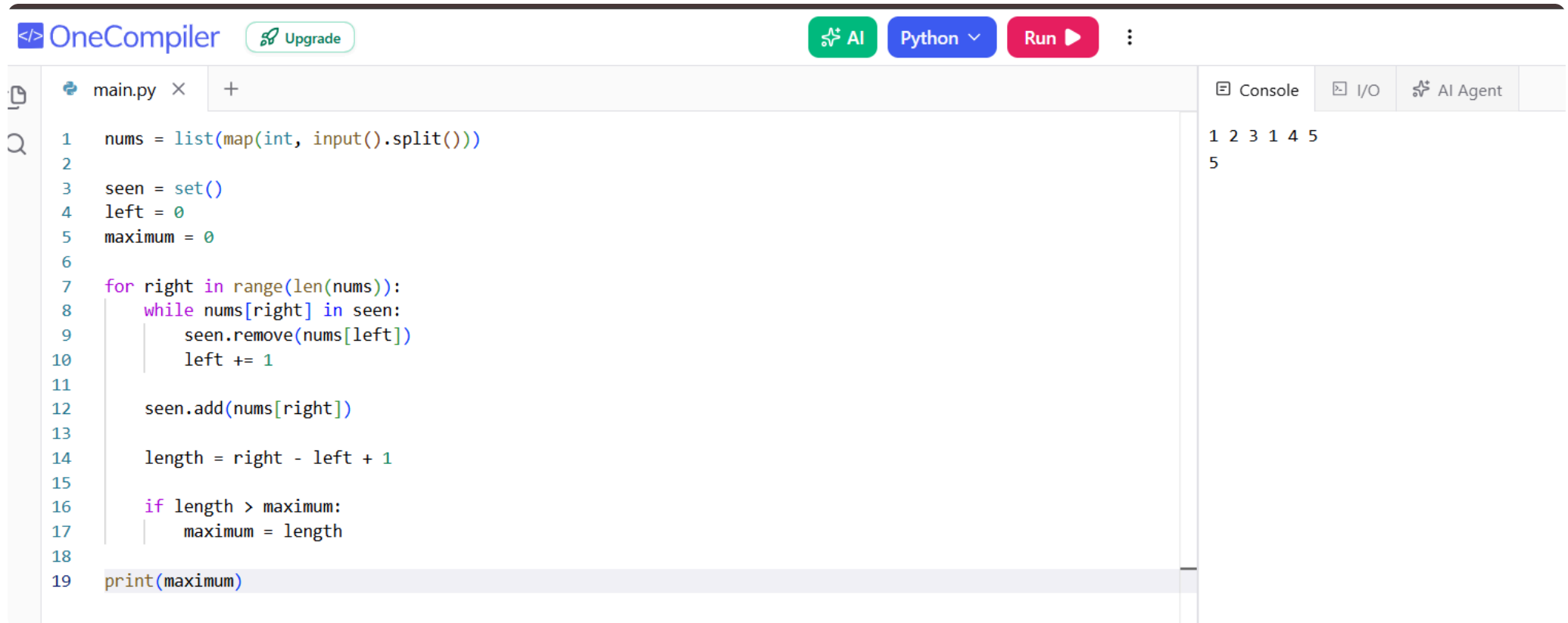


1. Student Attendance Analysis

A college maintains the daily attendance details of its students in the form of a list containing student IDs. Some students may have attended multiple sessions on the same day. The administration wants to identify the longest continuous sequence of sessions in which no student ID is repeated. Develop a solution that determines the maximum length of such a sequence.



The screenshot shows the OneCompiler Python IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. The main editor area displays a Python script in a file named 'main.py'. The script implements a sliding window algorithm to find the longest continuous sequence of sessions with no repeated student IDs. The input is a list of student IDs: [1, 2, 3, 1, 4, 5]. The output is 5, representing the length of the longest continuous sequence [2, 3, 1, 4, 5].

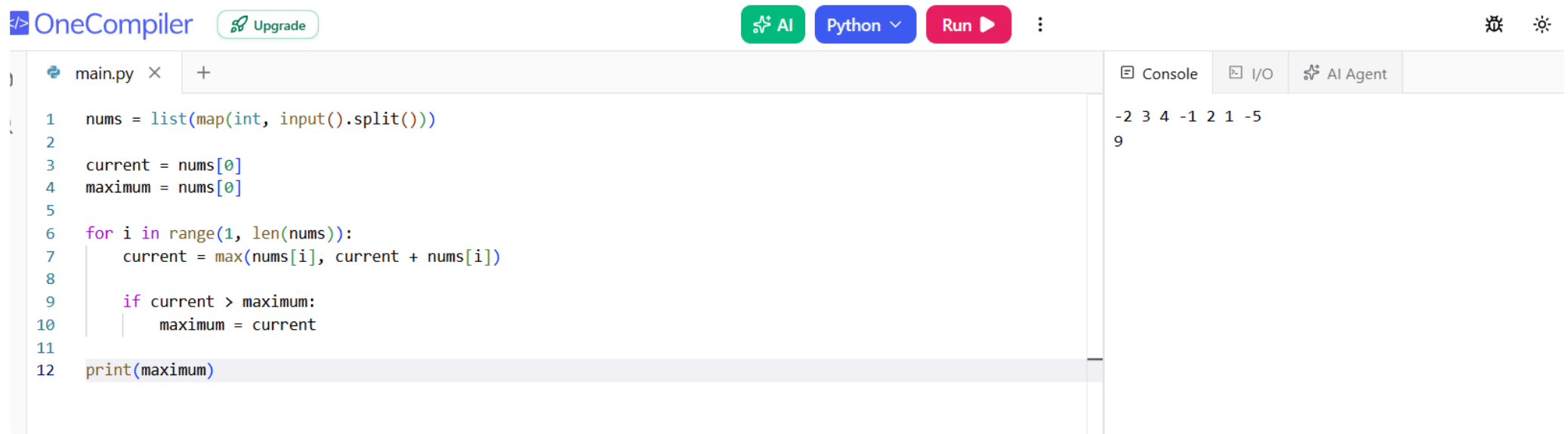
```
1  nums = list(map(int, input().split()))
2
3  seen = set()
4  left = 0
5  maximum = 0
6
7  for right in range(len(nums)):
8      while nums[right] in seen:
9          seen.remove(nums[left])
10         left += 1
11
12     seen.add(nums[right])
13
14     length = right - left + 1
15
16     if length > maximum:
17         maximum = length
18
19 print(maximum)
```

The right sidebar shows the 'Console' tab with the input and output:

```
1 2 3 1 4 5
5
```

2. Online Shopping Price Analysis

An online shopping application stores the prices of products viewed by a customer during a browsing session. The customer wants to identify a continuous range of products that provides the maximum possible total discount value. Given the discount values, determine the maximum value that can be obtained from any continuous range.



The screenshot shows the OneCompiler Python IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. The main editor area displays a Python script named 'main.py' with the following code:

```
1  nums = list(map(int, input().split()))
2
3  current = nums[0]
4  maximum = nums[0]
5
6  for i in range(1, len(nums)):
7      current = max(nums[i], current + nums[i])
8
9      if current > maximum:
10         maximum = current
11
12 print(maximum)
```

The right sidebar shows the 'Console' tab with the input and output:

```
-2 3 4 -1 2 1 -5
9
```

3. Rainwater Collection System

A city installs buildings of different heights along a straight road. During rainfall, water gets collected between taller buildings. The engineering team

needs to calculate the total amount of water that can remain trapped after heavy rainfall based on the heights of the buildings.



```
main.py x +
1 height = list(map(int, input().split()))
2
3 water = 0
4
5 for i in range(len(height)):
6     left_max = max(height[:i + 1])
7     right_max = max(height[i:])
8
9     trapped = min(left_max, right_max) - height[i]
10
11     if trapped > 0:
12         water += trapped
13
14 print(water)
```

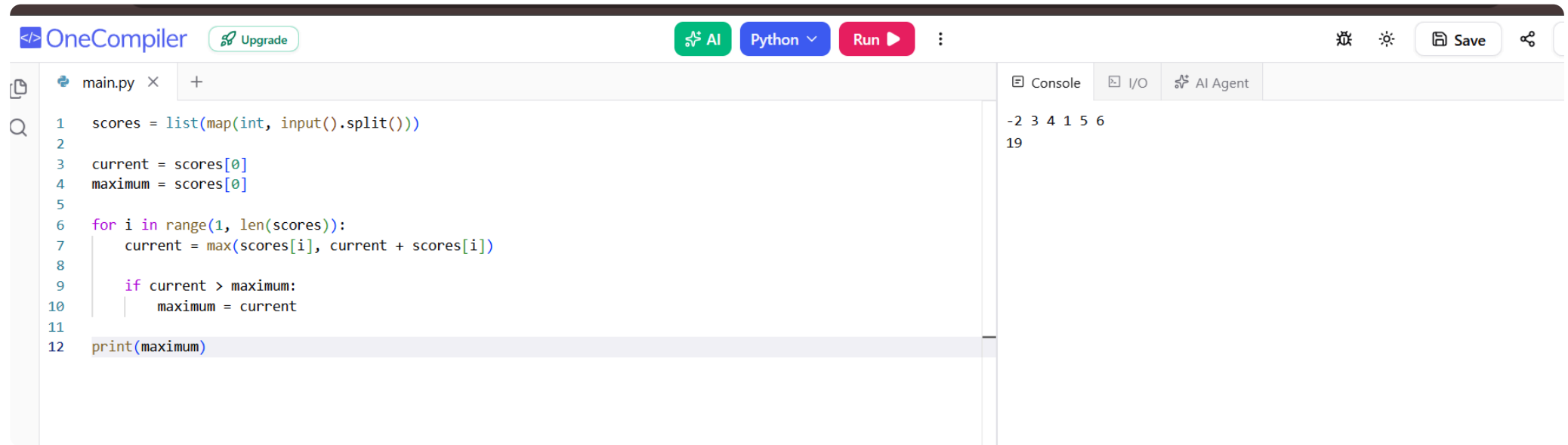
Console I/O

0 1 0 2 1 0 1 3
5

4. Employee Performance Analysis

A company stores the monthly performance scores of an employee for several months. The scores may contain both positive and negative values depending on the employee's performance. Management wants to identify the

continuous period during which the employee achieved the highest overall performance.



The screenshot shows the OneCompiler Python IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. The main editor area displays a Python script in a file named 'main.py'. The script implements a function to find the maximum sum of a contiguous subarray using a sliding window approach. The console on the right shows the input '-2 3 4 1 5 6' and the output '19'.

```
1 scores = list(map(int, input().split()))
2
3 current = scores[0]
4 maximum = scores[0]
5
6 for i in range(1, len(scores)):
7     current = max(scores[i], current + scores[i])
8
9     if current > maximum:
10         maximum = current
11
12 print(maximum)
```

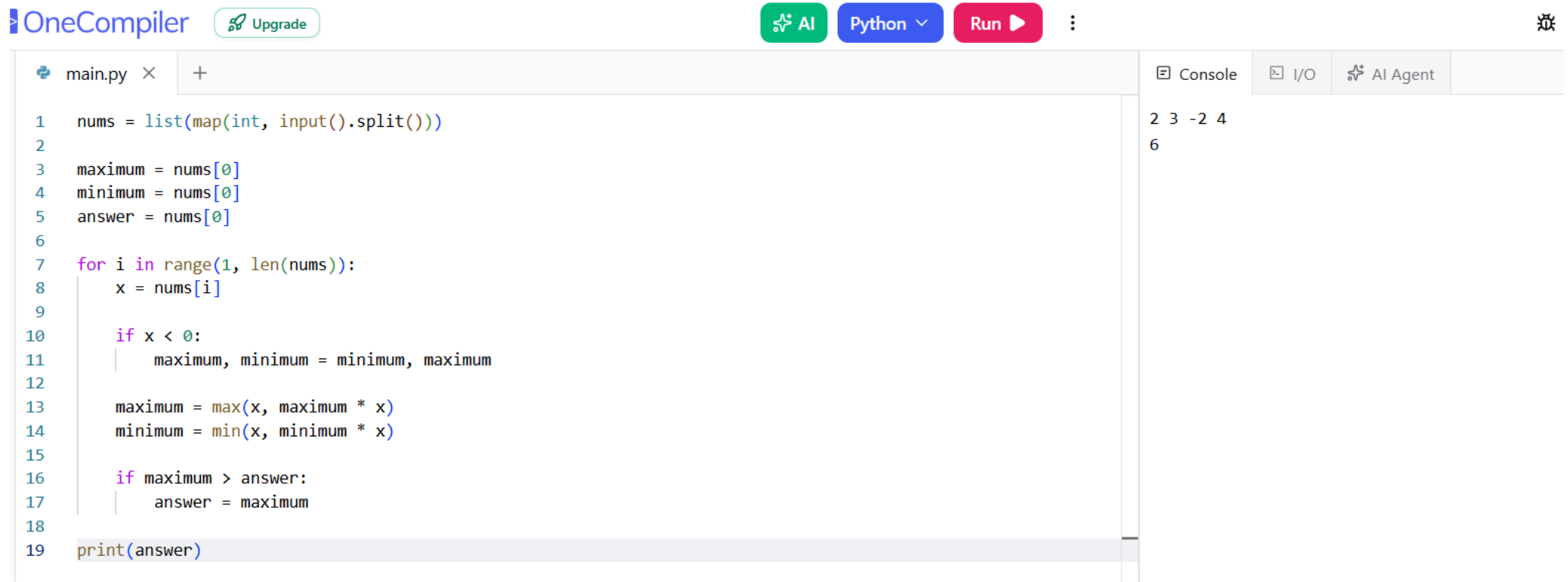
Console Output:

```
-2 3 4 1 5 6
19
```

5. Product Sales Analysis

A retail company stores the daily sales quantity of a product for several consecutive days. Due to seasonal changes, some days may have negative adjustments. The company wants to identify the period that produced the

highest multiplication of sales-related values. Develop a solution to determine this maximum product.



The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. Below the header, the editor displays a file named 'main.py' with the following Python code:

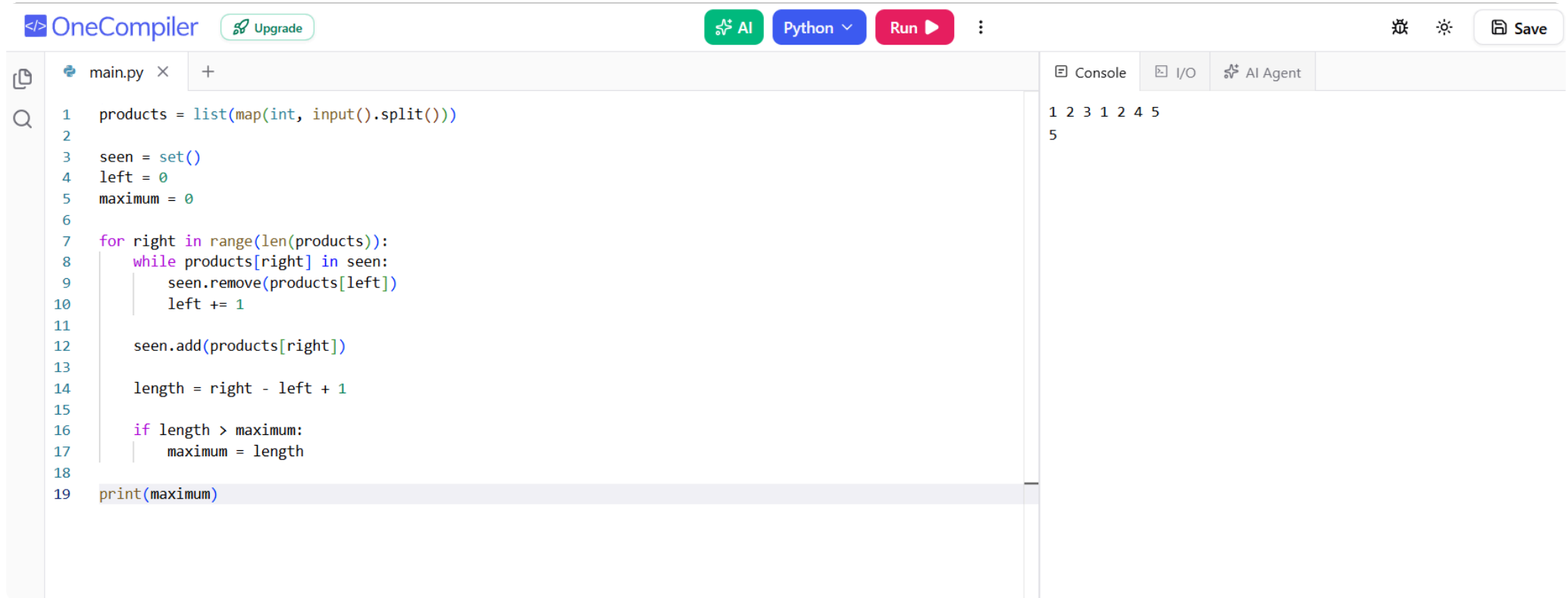
```
1  nums = list(map(int, input().split()))
2
3  maximum = nums[0]
4  minimum = nums[0]
5  answer = nums[0]
6
7  for i in range(1, len(nums)):
8      x = nums[i]
9
10     if x < 0:
11         maximum, minimum = minimum, maximum
12
13     maximum = max(x, maximum * x)
14     minimum = min(x, minimum * x)
15
16     if maximum > answer:
17         answer = maximum
18
19 print(answer)
```

On the right side, the 'Console' tab is active, showing the input '2 3 -2 4' and the output '6'. Other tabs for 'I/O' and 'AI Agent' are also visible.

6. Customer Purchase History

An e-commerce application stores the product IDs purchased by a customer in chronological order. The same product may appear multiple times. The system

needs to determine the longest sequence of consecutive purchases in which every product ID is unique.



The screenshot shows the OneCompiler Python IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. The main editor area displays a Python script for finding the longest sequence of unique product IDs. The script uses a sliding window approach with a set to track unique products. The console output shows the input sequence '1 2 3 1 2 4 5' and the resulting maximum length '5'.

```
1 products = list(map(int, input().split()))
2
3 seen = set()
4 left = 0
5 maximum = 0
6
7 for right in range(len(products)):
8     while products[right] in seen:
9         seen.remove(products[left])
10        left += 1
11
12    seen.add(products[right])
13
14    length = right - left + 1
15
16    if length > maximum:
17        maximum = length
18
19 print(maximum)
```

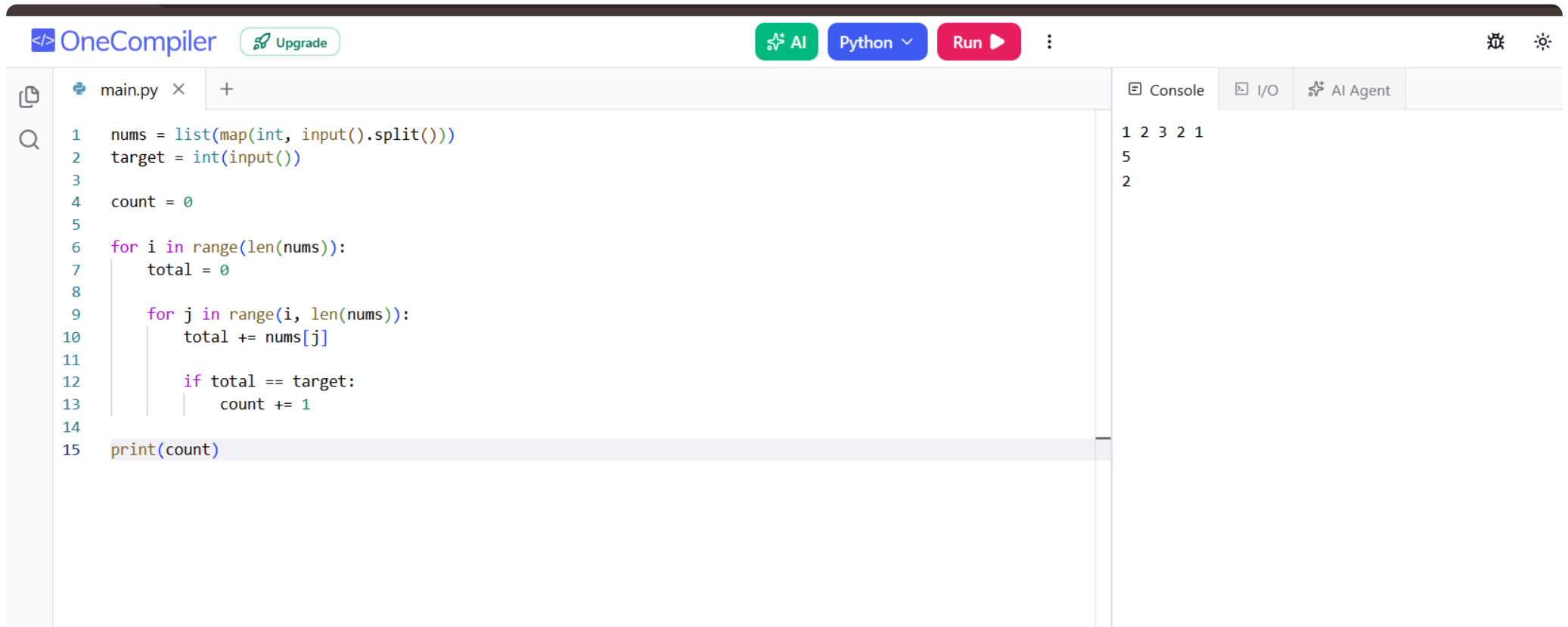
Console Output:

```
1 2 3 1 2 4 5
5
```

7. Bank Transaction Analysis

A bank stores transaction amounts for a customer's account. A continuous group of transactions may add up to a specific target amount. The auditing

system needs to determine how many different continuous transaction groups produce exactly the specified amount.



The screenshot shows the OneCompiler Python IDE interface. The main editor displays a Python script named `main.py` with the following code:

```
1  nums = list(map(int, input().split()))
2  target = int(input())
3
4  count = 0
5
6  for i in range(len(nums)):
7      total = 0
8
9      for j in range(i, len(nums)):
10         total += nums[j]
11
12         if total == target:
13             count += 1
14
15 print(count)
```

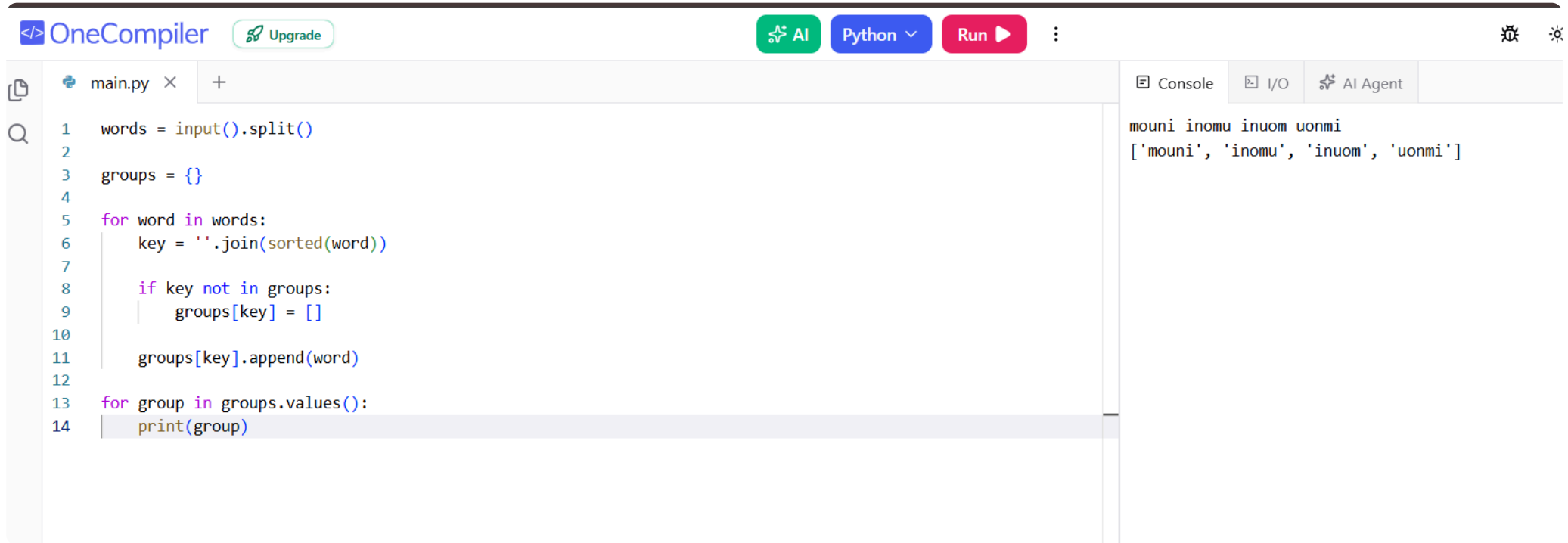
The right sidebar shows the Console output with the following text:

```
1 2 3 2 1
5
2
```

8. Employee Skill Grouping

A company receives a list of employee skill codes represented as strings. Employees having the same set of characters in their skill codes belong to the

same skill category, even if the characters appear in a different order. The HR system needs to organize employees into appropriate skill groups.



The screenshot shows the OneCompiler web IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. Below the header, the editor displays a Python script in a file named 'main.py'. The script is designed to take a list of words as input and group them into anagrams based on their sorted characters. The console on the right shows the output of the script, which is a list of lists, where each inner list contains words that are anagrams of each other.

```
1 words = input().split()
2
3 groups = {}
4
5 for word in words:
6     key = ''.join(sorted(word))
7
8     if key not in groups:
9         groups[key] = []
10
11     groups[key].append(word)
12
13 for group in groups.values():
14     print(group)
```

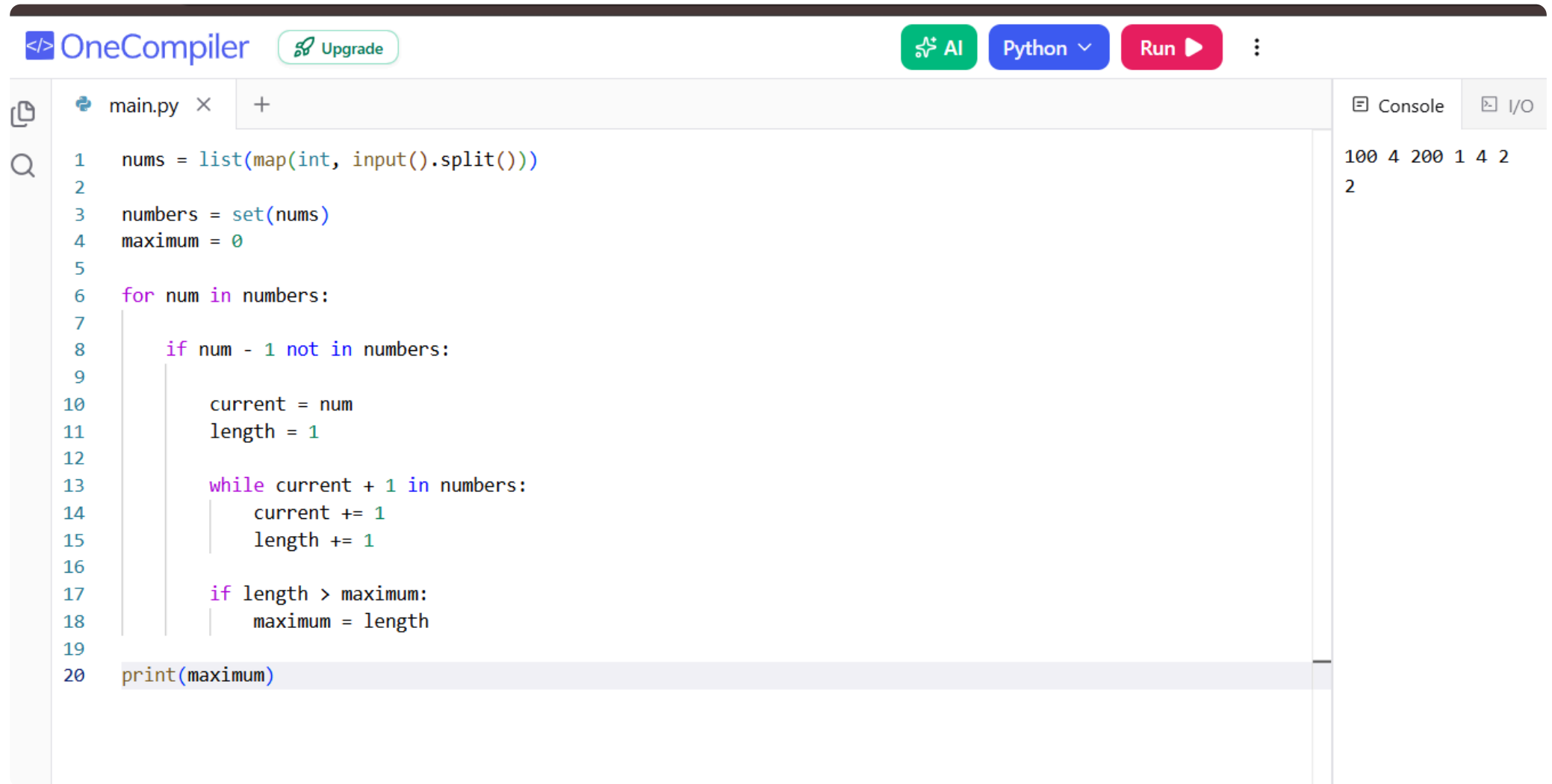
Console Output:

```
mouni inomu inuom uonmi
['mouni', 'inomu', 'inuom', 'uonmi']
```

9. Network Packet Analysis

A network monitoring system receives packet identifiers in chronological order. The system must determine the longest sequence of consecutive

packets whose identifiers form a continuous numerical sequence, regardless of their original order in the incoming data.



The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. Below the header, the editor displays a Python script in a file named 'main.py'. The script takes input from the user, converts it to a set of integers, and then iterates through the set to find the longest continuous sequence of numbers. The console on the right shows the output of the script, which is '100 4 200 1 4 2' followed by '2' on the next line, indicating the length of the longest continuous sequence.

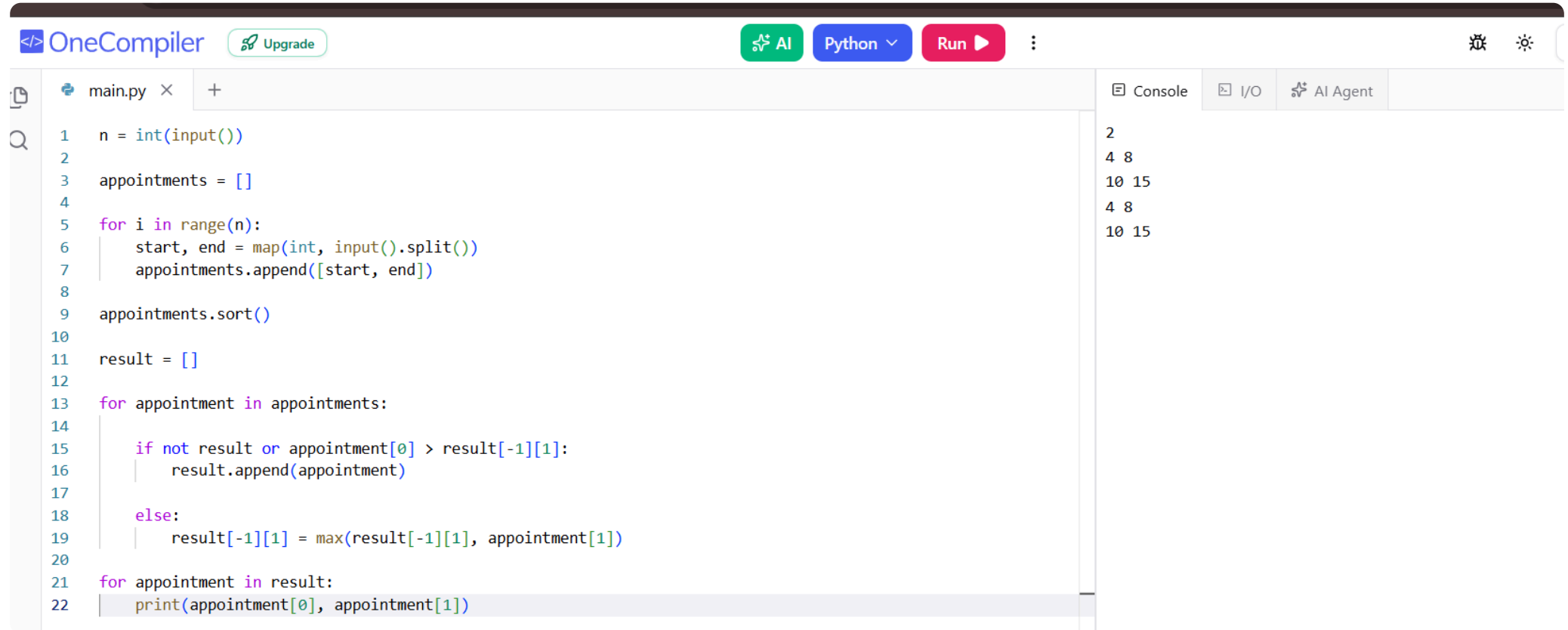
```
1 nums = list(map(int, input().split()))
2
3 numbers = set(nums)
4 maximum = 0
5
6 for num in numbers:
7     if num - 1 not in numbers:
8         current = num
9         length = 1
10
11         while current + 1 in numbers:
12             current += 1
13             length += 1
14
15         if length > maximum:
16             maximum = length
17
18 print(maximum)
```

Console Output:

```
100 4 200 1 4 2
2
```

10. Hospital Appointment Scheduling

A hospital receives appointment requests represented by starting and ending times. Some appointments overlap with each other. The scheduling system needs to combine overlapping appointment periods so that the final schedule contains only non-overlapping time ranges.



The screenshot shows the OneCompiler Python IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Python', and 'Run'. The main editor area displays a Python script named 'main.py' with the following code:

```
1 n = int(input())
2
3 appointments = []
4
5 for i in range(n):
6     start, end = map(int, input().split())
7     appointments.append([start, end])
8
9 appointments.sort()
10
11 result = []
12
13 for appointment in appointments:
14     if not result or appointment[0] > result[-1][1]:
15         result.append(appointment)
16
17     else:
18         result[-1][1] = max(result[-1][1], appointment[1])
19
20
21 for appointment in result:
22     print(appointment[0], appointment[1])
```

The right sidebar shows the 'Console' output, which displays the results of the program execution:

```
2
4 8
10 15
4 8
10 15
```